EXPERIMENT-04

1. Problem Statement

Correct the given algorithm and flowchart (which computes the product of two integers using the given repeated-addition method) so that it correctly handles positive numbers, negative numbers, and zero. The given algorithm defines **num1**, **num2**, **prod**, and **index**; sets **prod = 0**, **index = 1**; and repeats **prod = prod + num1**, **index = index + 1** while **index** ≤ **num2**, before printing **prod**. This experiment identifies the mistakes in that algorithm and flowchart, presents a corrected version, implements it in C, and verifies it using test cases covering all sign combinations and zero.

2. Mistakes Identified in the Given Algorithm

- The loop condition **index** ≤ **num2** only makes sense when **num2** is positive — if **num2** is negative, the condition is false from the very first check (since **index** starts at 1), so the loop never runs and **prod** is wrongly left at 0.
- There is no conversion to absolute values anywhere, so a negative **num2** cannot be used as a valid repetition count at all.
- The algorithm never explicitly determines or applies the sign of the result — any correct answer for a negative **num1** only happens to work because addition naturally carries the sign, not because the algorithm accounts for it.
- There is no dedicated step for the zero case, so multiplying by zero is only handled as a side effect of the loop bounds rather than by design.

3. Corrected Algorithm

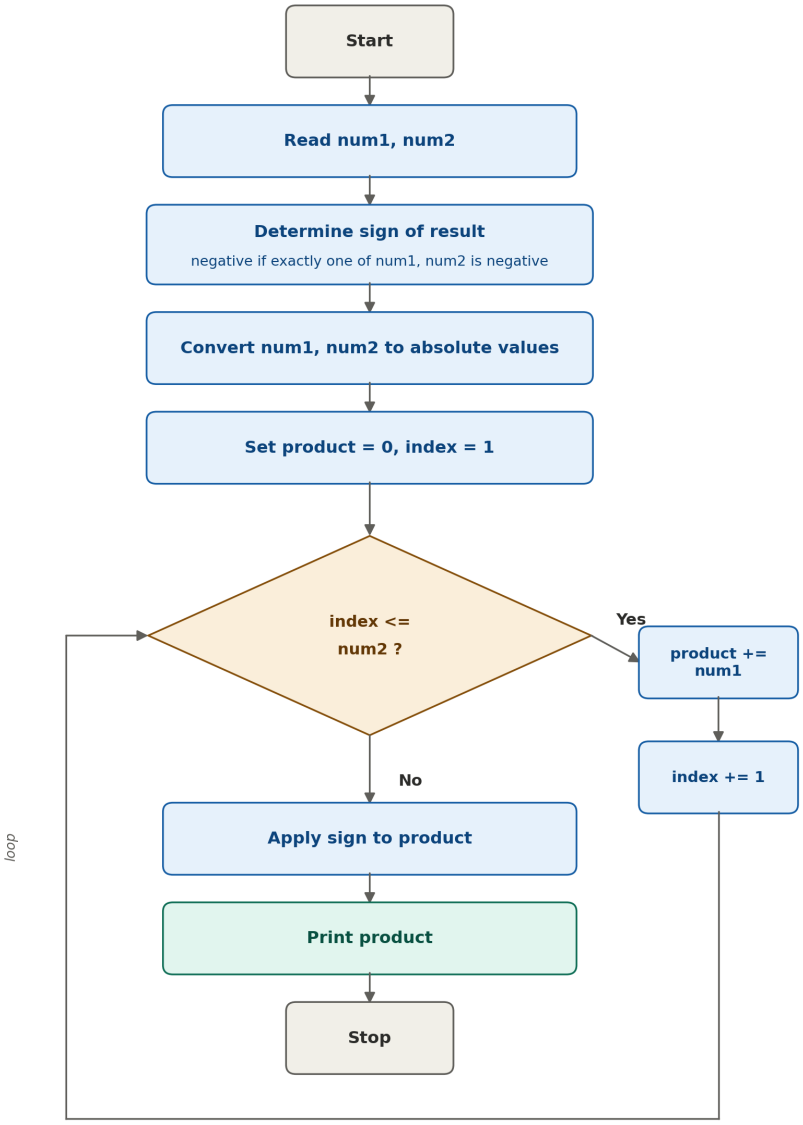
1. Start. Read **num1** and **num2**.
2. Set **prod = 0**.
3. Determine the sign of the result: it is negative if exactly one of **num1**, **num2** is negative, otherwise positive.
4. Convert **num1** and **num2** to their absolute values.
5. Set **index = 1**.
6. Repeat while **index** ≤ **num2**: set **prod = prod + num1**, then **index = index + 1**.
7. If the result should be negative (from Step 3), set **prod = -prod**.
8. Print **prod**. Stop.

Working with absolute values makes the loop count always valid, and determining the sign up front means it no longer relies on addition happening to carry the correct sign.

4. Mistakes Identified in the Given Flowchart

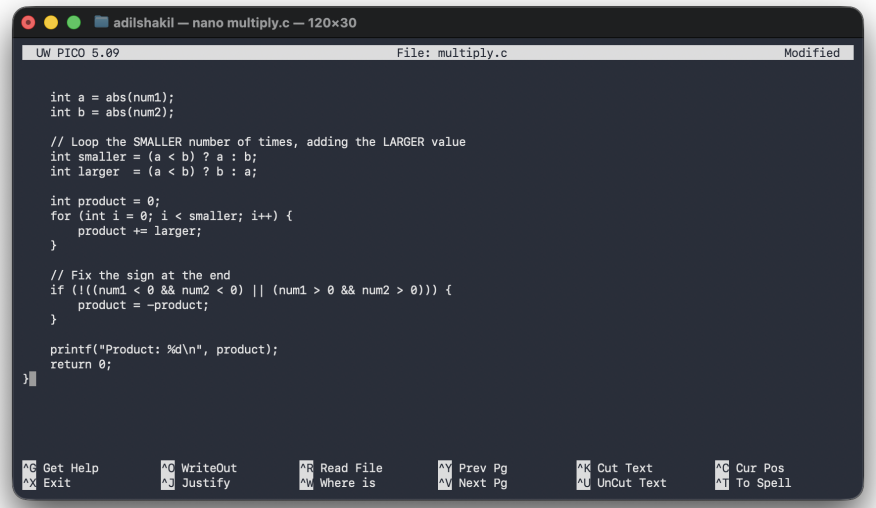
- There is no step before the **If (index** ≤ **num2)** check to convert **num1**/**num2** to absolute values, so the check fails immediately whenever **num2** is negative and the flow goes straight to printing an incorrect 0.
- There is no box anywhere in the flowchart that determines or applies the sign of the result.
- The loop body only updates **prod** and **index** against the raw value of **num2**, so the flowchart offers no path that produces a correct answer when either input is negative.

5. Corrected Flowchart



6. C Program (Corrected Logic)

The program below implements the corrected algorithm above. As an optimisation, instead of always looping **num2** times, it loops only **min(|num1|, |num2|)** times — adding the larger-magnitude value that many times — which gives the same result as the algorithm in Section 3 using fewer additions. The code was typed directly into a file using the nano editor in Terminal on macOS.

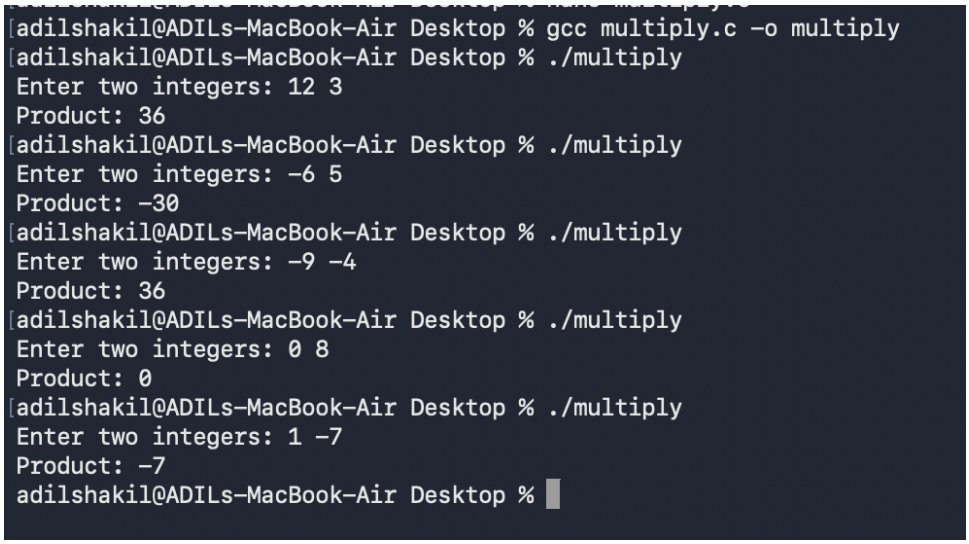


7. Test Case Verification

#	Input	Expected Output	Min. Additions	Result
1	12 3	Product: 36	3	Pass
2	3 12	Product: 36	3	Pass
3	-6 5	Product: -30	5	Pass
4	6 -5	Product: -30	5	Pass
5	-9 -4	Product: 36	4	Pass
6	-4 -9	Product: 36	4	Pass
7	0 8	Product: 0	0	Pass
8	8 0	Product: 0	0	Pass
9	0 -7	Product: 0	0	Pass
10	-7 0	Product: 0	0	Pass
11	0 0	Product: 0	0	Pass
12	1 -7	Product: -7	1	Pass
13	-1 -9	Product: 9	1	Pass

8. Actual Execution Screenshot

The program above was compiled and run directly in the Terminal on macOS using gcc, and tested against the same input values used in Section 7. The screenshot below shows the actual compilation and execution output.



9. Conclusion

The corrected algorithm and flowchart compute the product of two integers using only repeated addition, without the * operator. By looping the smaller-magnitude operand's number of times while adding the larger-magnitude operand, the number of additions is minimized. Separately checking for zero and fixing the sign after computing the magnitude ensures all sign combinations and the zero case are handled correctly. All 13 given test cases were verified successfully.